RATIONAL WEB

Technical Architecture & Implementation Specification

Provisional project name | Design baseline v0.1 | 22 October 2026

Status: proposed design, not an implemented or validated framework. This document consolidates agreed directions and explicitly identifies unresolved decisions.

Executive summary

Rational Web is a Python-first, object-oriented web framework designed to reduce the conceptual fragmentation of full-stack development. Applications, pages, components, services and models are represented as Python objects. FastAPI provides the HTTP/ASGI foundation; SQLAlchemy supplies persistence. Initial pages are server-rendered. A small browser runtime attaches interactive behavior and applies reactive updates. A typed, composable styling system compiles Python style objects into CSS, including responsive variants and global theme tokens.

The objective is not to abolish HTML, CSS, JavaScript or HTTP, but to make them implementation mechanisms behind a consistent, inspectable Python API. Security and network boundaries remain explicit.

1. Goals, principles and non-goals

1.1 Goals

- One coherent Python-facing application model with minimal boilerplate.
- High-quality responsive defaults, without requiring hand-written CSS or JavaScript for common tasks.
- SSR for the first page; local browser interactions where appropriate; server RPC for privileged operations.
- Strong typing, editor completion, testability, reproducibility and clear error messages.
- Optional escape hatches to FastAPI, SQLAlchemy, raw CSS and browser JavaScript.
- Publishable, installable library with stable documented contracts and examples.

1.2 Principles

- Composition over inheritance for styles and reusable behavior.
- Explicit remote exposure: Python methods are not automatically callable from browsers.
- Predictable execution: a developer can tell whether an operation is server-side or browser-side.
- Immutable style definitions and immutable normalized render nodes; state is explicitly mutable.
- Secure by default: session isolation, authentication hooks, output escaping and event authorization.
- Progressive enhancement: useful SSR markup should exist before browser code executes.
- Small stable core, optional adapters, and an inspectable generated output.

1.3 Non-goals for v0.1

No general Python-to-JavaScript compiler; no full React replacement; no automatic distributed-state consistency; no reimplementation of SQLAlchemy; no Bootstrap/Tailwind parity; no guarantee that arbitrary Python code can run in a browser.

2. Decisions and open questions

Area	Baseline decision	Status
Rendering	SSR first + selective hydration + DOM patching	Chosen direction
Client methods	Small declarative client operations; arbitrary Python executes on server	Chosen direction
Styling	Typed immutable Style, merged via composition	Chosen direction
Responsiveness	CSS viewport and container queries	Chosen direction
Backend	FastAPI + Pydantic + SQLAlchemy adapters	Chosen direction
Remote API	Explicit @remote; validated and authorized	Chosen direction
Global appearance	Semantic tokens + curated theme presets	Chosen direction
Client navigation	Full reload first; optional SPA navigation later	Deferred
State transport	HTTP event RPC first; WebSocket optional	Proposed
Browser diff	Server produces targeted patches; refine protocol later	Proposed
Component identity	Stable keyed paths and instance IDs	To validate

3. Public programming model

App owns configuration, routes, services, theme and infrastructure; Page owns request-scoped page behavior; Component encapsulates a subtree; Service exposes explicitly declared operations; Model represents persisted data. A developer-facing API should be ergonomic but never confuse page instances across users.

```
from rational import App, Page, Component, Text, Button, State, Style
from rational.backend import Service, remote

app = App(theme="modern", responsive="adaptive")

class Counter(Component):
    count = State(0, scope="component")

    def increment(self):
```

```
        self.count += 1

    def render(self):
        return [Text(lambda: f"Count: {self.count}"),
                Button("Increment", on_click=self.increment)]

@app.page("/")
class Home(Page):
    def render(self):
        return Counter(style=Style(padding=16))

app.run()
```

API examples are target syntax, not currently executable. In particular, binding syntax and state descriptors require prototyping. A callable text expression avoids incorrectly assuming an ordinary Python f-string will stay reactive after evaluation.

3.1 Ownership and lifecycle

- App: created at process startup; configuration and service registrations are long-lived. Mutable global state is not assumed to be consistent across worker processes.
- Page: created per request/render session, not as a process-global singleton. Route params and authenticated context are injected.
- Component: owned by a page or parent component; identity is derived from stable keys and parent identity.
- Service: injected with appropriate lifetime (application, request, transaction); no direct browser object reference.
- Model: SQLAlchemy-backed persistence; DB sessions are request/operation-scoped.

4. Rendering architecture

Request flow: GET route -> FastAPI handler -> page factory -> component tree -> normalized node tree -> HTML serializer + CSS compiler -> response with hydration manifest. Browser displays HTML and attaches handlers only to interactive islands.

Interactive flow: browser event -> local declarative operation OR authenticated event request -> server resolves component/session -> validates event and arguments -> invokes method -> rerenders affected subtree -> sends patch and state revision -> browser applies patch.

4.1 Intermediate representation

```
Node(tag="button", attrs={"type": "button"},
    children=(TextNode("Save"),), key="save")
# Nodes are immutable; all text/attributes are escaped at serialization.
```

The normalized node tree must separate structural HTML, event references, style hashes, stable keys, and accessibility attributes. It must reject unsafe attributes and script injection unless a deliberately unsafe escape hatch is invoked.

4.2 SSR, hydration and CSR

- SSR always generates meaningful initial HTML and critical CSS.

- Hydration manifest includes only public, JSON-serializable client state, component IDs, event descriptors and version information; secrets never appear.
- Static elements require no hydration. Interactive islands attach listeners without recreating the entire page.
- Browser-local state supports only explicitly supported operations in v0.1. Python callbacks otherwise execute on the server.
- Client navigation, streaming SSR and richer local rendering are later extensions, not prerequisites for the first release.

4.3 Patch protocol and failures

Define an explicit versioned protocol: event_id, page_session_id, component_id, event_name, validated payload, state_revision and CSRF token where applicable. Responses include revision, replacement/attribute/text patches or a full subtree replacement, plus structured errors. Reject stale revisions or reconcile them deterministically. Handle double clicks, retries, disconnections and navigation races. Use HTTP POST initially; WebSockets are optional.

5. State and reactivity

Scope	Ownership	Persistence
component	One mounted component	Until unmount / session end
page	One page session	Until navigation / session end
session	Authenticated or anonymous session	Session store with TTL
application	Process or external shared store	External store for multiworker consistency
persistent	Database model	Database

State descriptors should validate values and record writes. During render, dependency tracking can identify which components read which state cells. The initial implementation may rerender a whole affected component and diff immutable nodes. Batch updates per event; avoid unrestricted mutable nested objects that bypass change detection. Offer explicit set/update methods for complex state.

Do not conflate browser state with server state. Define serialization rules, initial ownership, update direction, conflict behavior and permissions for each scope. Never serialize server secrets or private objects into hydration data.

6. Services, HTTP and persistence

```
class UserService(Service):
  @remote(method="POST", auth="required")
  async def create_user(self, name: str) -> UserDTO:
    async with self.db.session() as session:
      user = User(name=name)
      session.add(user)
      await session.commit()
      return UserDTO.model_validate(user)
```

A @remote decorator registers a typed RPC endpoint and generates OpenAPI metadata. The framework validates input/output with Pydantic, applies authentication and per-method

authorization, translates exceptions into safe structured errors, and enforces request size and rate limits as configured. Default to POST for side-effecting RPC; explicit GET is reserved for safe idempotent reads. Avoid leaking internal stack traces.

SQLAlchemy remains the source of truth for ORM semantics. Provide model helpers and a request-scoped session factory, not a second query language. Explicitly document sync versus async engines, transaction boundaries, migrations (via Alembic), lazy loading and N+1 pitfalls. Consider SQLModel as an optional compatibility path rather than a mandatory abstraction.

7. Styling and design system

Style is an immutable typed value object; None means unspecified, not CSS none. Merge operator | performs a deterministic right-biased merge. Style compiler normalizes properties and units, resolves tokens, hashes declarations and emits deduplicated CSS classes. The same style object can be shared across components.

```
base = Style(color="on_primary", background="primary",
        padding=12, border_radius=8)
button_style = base | Style(font_weight=600)
responsive = Style(
   padding=24,
   viewport={"mobile": Style(padding=12)},
   container={"compact": Style(layout="vertical")},
   hover=Style(background="primary_hover"),
)
```

Define a canonical set of properties with explicit unit rules (e.g., numeric spacing values compile to px or a documented spacing scale). Provide escape hatches for custom CSS properties and raw CSS, with validation. Support pseudo-states such as hover/focus/disabled; focus-visible and reduced-motion must be part of accessible defaults.

7.1 Responsive behavior

Viewport breakpoints are mobile-first min-width CSS media queries; container variants use CSS container queries. Define a named breakpoint registry with defaults that applications can override. Breakpoint names are semantic conventions, not device detection. The browser handles resizing; Python must not inspect viewport dimensions to choose CSS. A container must explicitly establish container-type where required.

7.2 Themes and presets

A Theme provides semantic tokens: primary, on_primary, surface, on_surface, border, muted, spacing scale, radius scale, typography, shadows, density and motion. Themes compile to CSS custom properties and can switch between light/dark/system without rebuilding the page. Ship a small curated set: modern, minimal, soft and professional. Default components must be attractive and responsive without extra configuration.

7.3 Bootstrap/Tailwind

Adapters are deferred. Native Style is canonical; Bootstrap adapter maps variants to component classes; Tailwind adapter requires a compatible utility generation/build strategy. Do not promise exact equivalence between systems. Custom class names remain supported.

8. Layout and component library

Ship Text, Button, Link, Input, Form, Row, Column, Grid, Stack, Card and Image as basic primitives. Components render semantic HTML; Row/Column/Grid are ergonomic wrappers around flexbox/grid, not a proprietary layout engine. Provide explicit labels, keyboard behavior, form errors, ARIA where appropriate and semantic heading levels. Custom classes may implement render() and compose primitives.

9. Security, reliability and performance

- Default escape all text and attributes; sanitize or restrict raw HTML; establish Content Security Policy compatibility.
- Authenticate every remote request; authorize at service/method and resource level; protect cookies and CSRF-sensitive operations.
- Never trust event IDs, component IDs or client-supplied state; validate ownership against the session.
- Prevent cross-user state leakage and ensure isolation across concurrent requests and multiple workers.
- Bound server-held component state by TTL, quotas and eviction; design reconnection and graceful degradation.
- Cache style hashes and compiled CSS; minimize runtime JS and hydration manifest size.
- Support logging, request tracing, event tracing, debug mode and inspectable render/patch output.
- Measure first render latency, JS bundle size, CSS bytes, hydration time, event latency and server memory per active session.

10. Implementation roadmap and acceptance gates

0. 0. Architecture proof

Write two sample apps and formalize lifecycle, state scopes and wire protocol. Acceptance: no ambiguity about server/client execution.

1. 1. Project skeleton

Set up src package, pyproject, lint, type checking, CI, docs, tests and example app. Acceptance: clean pip install.

2. 2. Component IR

Implement Node/TextNode, keys, renderer and escaping. Acceptance: deterministic snapshots and safe HTML.

3. 3. App/Page routing

Integrate FastAPI route registration, dependency injection and page factories. Acceptance: SSR multipage app.

4. 4. Style compiler

Implement immutable Style, merge, tokens, hashes, deduplication, pseudo-states and responsive CSS. Acceptance: CSS snapshots + browser viewport tests.

5. 5. Themes and primitives

Ship tokens, four presets, layout primitives, buttons and forms. Acceptance: responsive example without custom CSS.

6. 6. Event RPC

Add @remote/event registration, validation, session ownership, auth hooks, errors and HTTP transport. Acceptance: secure interactive counter and CRUD.

7. 7. Reactive rerender

Track state, render affected subtrees, compute/apply patches, handle revisions. Acceptance: independent users and concurrent events.

8. 8. Database integration

SQLAlchemy sessions, migrations guidance, models, transaction helpers and service injection. Acceptance: multiuser CRUD sample.

9. 9. Browser runtime

Ship tiny runtime, selective hydration, local declarative operations, reconnect handling. Acceptance: interaction without full reload.

10. 10. Hardening

Security review, load tests, accessibility, browser matrix, observability and documentation. Acceptance: public alpha release gate.

11. 11. Publish

Build wheel/sdist, TestPyPI, clean-install smoke test, semantic versioning, changelog and PyPI release.

11. Test strategy

Unit: style merging, unit normalization, CSS hashing, escaping, route registration, validation, state dependencies, patch algorithms. Integration: ASGI requests, database transactions, authentication, session isolation. Browser: Playwright on Chromium/Firefox/WebKit, viewport/container breakpoints, keyboard navigation, hydration and patching. Property-based tests: render determinism, patch correctness, event authorization. Load: simultaneous sessions, worker restarts, slow networks, rapid event bursts.

12. Repository, distribution and documentation

```
rational-web/
  pyproject.toml
```

```
src/rational/{app.py,page.py,component.py,state.py,events.py}
src/rational/rendering/{nodes.py,html.py,diff.py}
src/rational/styling/{style.py,compiler.py,theme.py,responsive.py}
src/rational/backend/{fastapi.py,services.py,database.py}
src/rational/static/runtime.js
tests/{unit,integration,browser}/
examples/{hello,counter,crud,dashboard}/
docs/{quickstart,concepts,api,security,migration}/
```

Document: installation, five-minute quickstart, conceptual model, SSR lifecycle, client/server boundary, state scopes, styling and themes, remote methods, database sessions, deployment, debugging, limitations and API reference. Publish typed package metadata (py.typed), wheel/sdist and license. Validate package name before registration. Support documented Python and browser versions and use CI release automation.

13. Risks, mitigations and scope discipline

Risk	Mitigation
Python method appears local but executes remotely	Explicit @remote, execution indicators and tracing
State leaks across users	Request/session ownership and isolation tests
SSR and hydration disagree	Deterministic IR, stable keys, serialized manifest tests
Complex CSS abstraction	Small typed core + raw CSS escape hatch
RPC attack surface	Auth, validation, CSRF, ownership, rate limits
Runtime becomes a JS framework	Limit browser runtime to event delegation and patches
Premature adapter compatibility	Defer Bootstrap/Tailwind until native API stabilizes
Too much implicit magic	Debug inspectability and explicit lifecycle docs

14. Release criteria and success metrics

Alpha acceptance: installable Python package; responsive themed SSR pages; reusable typed components; functional button/form events; secure multiuser CRUD with SQLAlchemy; no handwritten JavaScript required for demo; documented escape hatches; passing automated security and browser tests. Track first-render latency, event round-trip p50/p95, client bundle size, generated CSS size, memory/session, accessibility violations and time-to-first-working-app.

15. Recommended first engineering sprint

Day 1–2: specify Node and Style schemas and author the intended hello-world API. Day 3–4: implement component-to-node conversion and safe HTML serializer. Day 5: integrate FastAPI route and serve first SSR page. Day 6–7: implement Style-to-CSS compilation, hashing and responsive viewport variants. Day 8: create Button/Text/Row/Column primitives. Day 9: build a responsive themed landing page. Day 10: add tests, docs and a minimal release candidate. Only then start event RPC and reactivity.

16. Decisions requiring prototypes

- State descriptor semantics: instance defaults, inheritance, nested mutations and typing.
- Component identity and keyed reconciliation under conditional trees.
- Whether server events rerender a subtree or generate fine-grained patches.
- HTTP-only event RPC versus optional WebSocket sessions.
- Hydration manifest format and declarative browser-local operations.
- Database model API: direct SQLAlchemy models versus optional SQLModel.
- Style property naming, unit conventions and precedence between themes, variants and user overrides.
- Accessibility guarantees for higher-level components and form controls.

Recommendation: validate these questions with a counter, responsive dashboard and authenticated CRUD application before freezing the public API. Preserve room for evolution by marking the first release as 0.x.